COMPREHENSIVE DSA PATTERNS & APPROACHES CHEAT SHEET — Problem-Solving Blueprints & Complexities

```java
// 1A. Two Pointers (Sorted Arrays, Pairs, Palindromes) - O(N) Time | O(1) Space
int l = 0, r = nums.length - 1;
while (l < r) {
    int sum = nums[l] + nums[r];
    if (sum == target) return new int[]{l, r};
    else if (sum < target) l++;
    else r--;
}

// 1B. Fast & Slow Pointers (Floyd's Cycle Detection) - O(N) Time | O(1) Space
ListNode slow = head, fast = head;
while (fast != null && fast.next != null) {
    slow = slow.next;
    fast = fast.next.next;
    if (slow == fast) return true; // Cycle detected
}
```

Listing 1: 1. TWO POINTERS FAST/SLOW POINTERS

```java
// 2A. Fixed Window (Size K) - O(N) Time | O(1) Space
int windowSum = 0, maxSum = 0;
for (int i = 0; i < nums.length; i++) {
    windowSum += nums[i];
    if (i >= k - 1) {
        maxSum = Math.max(maxSum, windowSum);
        windowSum -= nums[i - (k - 1)]; // Remove leftmost
    }
}

// 2B. Variable Window (Dynamic Condition) - O(N) Time | O(K) Space
Map<Character, Integer> map = new HashMap<>();
int l = 0, maxLen = 0;
for (int r = 0; r < s.length(); r++) {
    map.put(s.charAt(r), map.getOrDefault(s.charAt(r), 0) + 1);
    while (map.size() > k) { // Invalid condition
        char leftChar = s.charAt(l);
        map.put(leftChar, map.get(leftChar) - 1);
        if (map.get(leftChar) == 0) map.remove(leftChar);
        l++;
    }
    maxLen = Math.max(maxLen, r - l + 1);
}
```

Listing 2: 2. SLIDING WINDOW (FIXED VARIABLE)

```java
// 3A. Classic Binary Search - O(log N) Time | O(1) Space
int l = 0, r = nums.length - 1;
while (l <= r) {
    int m = l + (r - l) / 2;
    if (nums[m] == target) return m;
    if (nums[m] < target) l = m + 1;
    else r = m - 1;
}

// 3B. Binary Search on Answer / Boundary - O(N log(Range))
int l = minPossible, r = maxPossible, ans = -1;
while (l <= r) {
    int m = l + (r - l) / 2;
    if (isValid(m)) {
        ans = m;        // Record valid answer
        r = m - 1;      // Try for smaller/larger optimal
    } else {
        l = m + 1;
    }
}
```

Listing 3: 3. BINARY SEARCH VARIANTS

```java
// 4A. Monotonic Stack (Next Greater Element) - O(N) Time | O(N) Space
int[] res = new int[n];
Arrays.fill(res, -1);
Deque<Integer> st = new ArrayDeque<>(); // Stores indices
for (int i = 0; i < n; i++) {
    while (!st.isEmpty() && nums[st.peek()] < nums[i]) {
        res[st.pop()] = nums[i];
    }
    st.push(i);
}

// 4B. Sliding Window Maximum (Deque) - O(N) Time | O(K) Space
Deque<Integer> dq = new ArrayDeque<>();
for (int i = 0; i < n; i++) {
    while (!dq.isEmpty() && dq.peekFirst() <= i - k) dq.pollFirst();
    while (!dq.isEmpty() && nums[dq.peekLast()] <= nums[i]) dq.pollLast();
    dq.offerLast(i);
    if (i >= k - 1) res[i - k + 1] = nums[dq.peekFirst()];
}
```

Listing 4: 4. MONOTONIC STACK SLIDING WINDOW DEQUE

```java
// 5A. Tree/Graph BFS (Level Order) - O(V+E) Time | O(V) Space
Queue<TreeNode> q = new LinkedList<>();
if (root != null) q.offer(root);
while (!q.isEmpty()) {
    int sz = q.size();
    for (int i = 0; i < sz; i++) {
        TreeNode curr = q.poll();
        if (curr.left != null) q.offer(curr.left);
        if (curr.right != null) q.offer(curr.right);
    }
}

// 5B. DFS (Pre/In/Post Order, Path Sums) - O(V+E) Time | O(H) Space
void dfs(TreeNode node, List<Integer> path) {
    if (node == null) return;
    path.add(node.val);
    if (node.left == null && node.right == null) { /* Base Case */ }
    dfs(node.left, path);
    dfs(node.right, path);
    path.remove(path.size() - 1); // Backtrack
}
```

Listing 5: 5. TREE TRAVERSALS BFS / DFS

```java
// 6A. Topological Sort (Kahn's Algorithm - BFS) - O(V+E)
int[] inDegree = new int[V];
for (int[] e : edges) inDegree[e[1]]++;
Queue<Integer> q = new LinkedList<>();
for (int i = 0; i < V; i++) if (inDegree[i] == 0) q.offer(i);
while (!q.isEmpty()) {
    int u = q.poll();
    for (int v : adj.get(u)) {
        if (--inDegree[v] == 0) q.offer(v);
    }
}

// 6B. Dijkstra's Shortest Path - O((V+E) log V) Time
PriorityQueue<int[]> pq = new PriorityQueue<>((a,b)->a[1]-b[1]);
pq.offer(new int[]{src, 0}); // {node, dist}
Arrays.fill(dist, Integer.MAX_VALUE);
dist[src] = 0;
while (!pq.isEmpty()) {
    int[] curr = pq.poll();
    int u = curr[0], d = curr[1];
    if (d > dist[u]) continue;
    for (int[] neighbor : adj.get(u)) {
        int v = neighbor[0], w = neighbor[1];
        if (dist[u] + w < dist[v]) {
            dist[v] = dist[u] + w;
            pq.offer(new int[]{v, dist[v]});
        }
    }
}
```

Listing 6: 6. GRAPH ALGORITHMS

```java
// Union-Find with Path Compression & Rank - O(\alpha(N)) Amortized
class DSU {
    int[] parent, rank;
    public DSU(int n) {
        parent = new int[n]; rank = new int[n];
        for (int i = 0; i < n; i++) parent[i] = i;
    }
    public int find(int i) {
        if (parent[i] == i) return i;
        return parent[i] = find(parent[i]); // Path compression
    }
    public boolean union(int i, int j) {
        int rootI = find(i), rootJ = find(j);
        if (rootI == rootJ) return false;
        if (rank[rootI] < rank[rootJ]) parent[rootI] = rootJ;
        else if (rank[rootI] > rank[rootJ]) parent[rootJ] = rootI;
        else { parent[rootJ] = rootI; rank[rootI]++; }
        return true;
    }
}
```

Listing 7: 7. DISJOINT SET UNION (UNION-FIND)

```java
// 8A. Top-K Frequent Elements - O(N log K) Time | O(N) Space
Map<Integer, Integer> count = new HashMap<>();
// ... populate frequency map ...
PriorityQueue<Integer> minH =
    new PriorityQueue<>((a, b) -> count.get(a) - count.get(b));
for (int num : count.keySet()) {
    minH.offer(num);
    if (minH.size() > k) minH.poll();
}

// 8B. Merge K Sorted Lists - O(N log K) Time | O(K) Space
PriorityQueue<ListNode> pq = new PriorityQueue<>((a,b)->a.val-b.val);
for (ListNode node : lists) if (node != null) pq.offer(node);
ListNode dummy = new ListNode(0), tail = dummy;
while (!pq.isEmpty()) {
    ListNode minNode = pq.poll();
    tail.next = minNode;
    tail = tail.next;
    if (minNode.next != null) pq.offer(minNode.next);
}
```

Listing 8: 8. TOP-K MERGE K LISTS (HEAPS)

```java
// Combinations / Permutations / Subsets - O(2^N) or O(N!) Time
void backtrack(int start, List<Integer> current, int[] nums) {
    res.add(new ArrayList<>(current)); // Process state
    for (int i = start; i < nums.length; i++) {
        // Optional: Skip duplicates -> if (i > start && nums[i] == nums[i-1])
            continue;
        current.add(nums[i]);              // Make decision
        backtrack(i + 1, current, nums); // Explore
        current.remove(current.size() - 1); // Undo decision
    }
}
```

Listing 9: 9. BACKTRACKING TEMPLATE

```java
// 10A. 0/1 Knapsack (1D Space Optimization) - O(N*W) Time | O(W) Space
int[] dp = new int[W + 1];
for (int i = 0; i < n; i++) {
    for (int w = W; w >= weight[i]; w--) { // Reverse for 0/1
        dp[w] = Math.max(dp[w], val[i] + dp[w - weight[i]]);
    }
}

// 10B. Unbounded Knapsack / Coin Change - O(N*W) Time | O(W) Space
for (int i = 0; i < n; i++) {
    for (int w = weight[i]; w <= W; w++) { // Forward for Unbounded
        dp[w] = Math.max(dp[w], val[i] + dp[w - weight[i]]);
    }
}

// 10C. Longest Common Subsequence (LCS) - O(M*N) Time | O(M*N) Space
int[][] dp = new int[m + 1][n + 1];
for (int i = 1; i <= m; i++) {
    for (int j = 1; j <= n; j++) {
        if (s1.charAt(i-1) == s2.charAt(j-1)) dp[i][j] = 1 + dp[i-1][j-1];
        else dp[i][j] = Math.max(dp[i-1][j], dp[i][j-1]);
    }
}
```

Listing 10: 10. DYNAMIC PROGRAMMING PATTERNS

```java
// 11A. Essential Bit Tricks - O(1) Time
n & (n - 1)          // Clears lowest set bit (Power of 2 check)
n & -n               // Extracts lowest set bit
a ^ b                // XOR: Bitwise difference, a^a=0, a^0=a

// 11B. Trie (Prefix Tree) Insert & Search - O(L) Time | O(N*L) Space
class TrieNode {
    TrieNode[] children = new TrieNode[26];
    boolean isWord = false;
}
void insert(String word) {
    TrieNode curr = root;
    for (char c : word.toCharArray()) {
        if (curr.children[c - 'a'] == null)
            curr.children[c - 'a'] = new TrieNode();
        curr = curr.children[c - 'a'];
    }
    curr.isWord = true;
}
```

Listing 11: 11. BIT MANIPULATION TRIE